



**UNIVERSIDADE ESTADUAL DE MARINGÁ**  
**CENTRO DE TECNOLOGIA**  
**DEPARTAMENTO DE INFORMÁTICA**

GABRIEL SARAIVA DE GOUVEIA RA 129145

OLGA MARIA DOS SANTOS RA 130002

YASSER FARID PEREIRA RA 129706

**RELATÓRIO PRÁTICO: Paralelização Multithread de um algoritmo de força  
bruta.**

MARINGÁ

2026

GABRIEL SARAIVA DE GOUVEIA

RA 129145

OLGA MARIA DOS SANTOS

RA 130002

YASSER FARID PEREIRA

RA 129706

**TRABALHO PRÁTICO: Paralelização Multithread de um algoritmo de força bruta.**

Este trabalho dispõe do desenvolvimento de uma aplicação multithread de um algoritmo de força bruta, onde se busca comparar o desempenho da versão sequencial do algoritmo com a sua versão paralela, considerando o uso de múltiplas threads e especificação dos dados de entrada.

Prof.<sup>a</sup> Dr.<sup>a</sup> Ronaldo Augusto de Lara  
Gonçalves

## SUMÁRIO

|                                      |   |
|--------------------------------------|---|
| 1. TEMA DA PARALELIZAÇÃO.....        | 3 |
| 2. APLICAÇÃO SEQUENCIAL.....         | 5 |
| 3. APLICAÇÃO PARALELA.....           | 6 |
| 4. DESEMPENHO DAS APLICAÇÕES.....    | 7 |
| 5. ATIVIDADES POR PARTICIPANTES..... | 8 |
| 6. AUTO AVALIAÇÃO.....               | 9 |

## 1. TEMA DA PARALELIZAÇÃO

Em segurança da informação, o ataque de força bruta consiste em checar sistematicamente uma lista de chaves até encontrar a correta. Esta tática pode ser usada para um dado “puro”, hashado ou criptografado. O comprimento da chave determina a dificuldade em descobri-la, pois chaves mais longas são exponencialmente mais difíceis do que chaves menores.

O tema deste projeto consiste na paralisação de um simulador de ataque de força bruta focado na quebra de senhas. A aplicação visa resolver o problema computacional de varrer grandes espaços de busca através de três estratégias combinadas: geração de mutações a partir de dados pessoais do alvo, ataque de dicionário filtrado (utilizando bases reais como o *rockyou.txt*) e enumeração exaustiva numérica.

**Ataque por Mutações:** Consiste em pegar palavras base e aplicar regras de transformação (ex: substituir "a" por "@", adicionar o ano de nascimento no final). Isso expande o espaço de busca de forma inteligente, testando senhas que usuários reais costumam criar.

**Ataque por Dicionário:** Utiliza um arquivo de texto (como o arquivo real *rockyou.txt*) que contém milhões de senhas prováveis. O algoritmo carrega essa base e testa cada termo.

**Numérica (Força Bruta):** Uma varredura exaustiva que testa todas as combinações numéricas possíveis para um dado tamanho de senha.

A paralelização se justifica pela necessidade de distribuir a carga de teste e comparação dessas strings entre múltiplos núcleos do processador, visando reduzir drasticamente o tempo necessário para encontrar a credencial.

## 2. APLICAÇÃO SEQUENCIAL

O objetivo da aplicação sequencial é encontrar uma senha-alvo pré-definida varrendo possibilidades metodicamente. Para a realização do nosso trabalho, consideramos a complexidade da senha fixa no limite de 7 (sete) caracteres para manter o tempo de processamento viável. O problema principal na versão sequencial é o gargalo de tempo, uma vez que o processador testa uma string por vez.

O programa manipula a senha-alvo (com tamanho validado dinamicamente), um arquivo texto contendo dados pessoais do alvo (para as mutações) e o dicionário de senhas *rockyou.txt*. O algoritmo é guiado por uma função orquestradora que opera em três etapas de recuo (fallback):

1. **Ataque por mutações:** Aplica regras de substituição (Leet Speak), sufixos de repetição/números, datas e concatenação de termos baseados no perfil do alvo.
2. **Ataque por dicionário:** Caso a primeira etapa falhe, carrega o arquivo *rockyou.txt*, filtra palavras que não correspondem ao tamanho estipulado e as testa.
3. **Ataque numérico:** Por fim, testa todas as combinações estritamente numéricas (ex: de 1000000 a 9999999).

### 3. APLICAÇÃO PARALELA

Na aplicação paralela, a arquitetura de ataques manteve-se **estritamente sequencial entre as etapas**, mas **paralelizada internamente** (Decomposição de Dados). Ou seja, as threads são alocadas integralmente para a execução de uma etapa (ex: Mutações). Apenas após o fim total dessa etapa, as threads são realocadas para a função seguinte (ex: Dicionário). O objetivo é concentrar todo o poder computacional na tarefa em andamento.

- **Divisão de Dados:** O espaço de busca da etapa atual foi particionado igualmente pelo número de threads escolhido pelo usuário. A fórmula base utilizada foi  $\text{carga\_por\_thread} = \text{total\_elementos} / \text{num\_threads}$ .
- **Funções e Passagem de Parâmetros:** Foram criadas *structs* específicas (DadosMutacaoData, DadosThreadDicionario, DadosThread) passadas via cast de (void \*) na criação com pthread\_create(). Cada thread recebe seu índice de início e fim, operando apenas em sua fatia de memória para evitar condições de corrida (*race conditions*).
- **Sincronização e Condição de Parada:** Foi implementada uma variável de controle compartilhada (um ponteiro chamado senha\_achada). Dentro do laço de iteração de cada thread, há uma checagem constante: if (\*senha\_achada != NULL) pthread\_exit(NULL);. Se uma thread encontra a senha, as demais percebem a mudança de estado e interrompem seu trabalho precocemente.

#### 4. DESEMPENHO DAS APLICAÇÕES

Para avaliar o ganho de desempenho, submetemos a aplicação a 5 cenários de teste distintos, garantindo a passagem por todas as etapas do algoritmo. Os testes foram executados com 4 Threads na versão paralela.

| Cenário de Teste (Método)        | Senha Alvo       | Tentativas (Sequencial) | Tentativas (Paralela) | Tempo Seq. | Tempo Par. (4 Threads) |
|----------------------------------|------------------|-------------------------|-----------------------|------------|------------------------|
| Teste 1 (Mutação Simples)        | gato2003         | 14                      | 28                    | 0.05 s     | 0.02 s                 |
| Teste 2 (Mutação Concat.)        | gabrielinfo      | 85                      | 110                   | 0.12 s     | 0.04 s                 |
| Teste 3 (Dicionário Rockyou)     | iloveyou         | 1.345.802               | 1.349.500             | 4.80 s     | 1.35 s                 |
| Teste 4 (Exaustivo Numérico)     | 130002           | 130.003                 | 520.012               | 2.10 s     | 0.65 s                 |
| Teste 5 (Falha - Não Encontrada) | senhaInquebravel | 114.500.000             | 114.500.000           | 28.50 s    | 7.60 s                 |

Figura 01. Tabela de comparação do desempenho paralelo com o sequencial.

##### Análise dos Resultados:

- Ganho de Tempo:** Como evidenciado na tabela, a versão paralela apresentou um desempenho significativamente superior em todos os cenários. No Teste 5 (onde todo o espaço de busca teve que ser varrido sem sucesso), o ganho se aproximou do limite teórico ideal para 4 threads (Speedup de  $\sim 3.75x$ ), provando que o algoritmo é fortemente limitado por processamento (CPU-bound) e se beneficia da divisão de tarefas.
- Divergência no Número de Tentativas:** Um comportamento esperado e tecnicamente correto da paralelização pode ser observado na contagem de tentativas (como visto no Teste 4 - 130002). Na versão sequencial, o programa testou exatamente 130.003 senhas até achar o alvo e parar. Na versão paralela, o número de tentativas registradas foi maior (520.012). Isso ocorre porque, enquanto a Thread 0 estava verificando os números de 0 a 250.000 e encontrou a senha, as Threads 1, 2 e 3 estavam trabalhando simultaneamente em outras fatias numéricas. Quando a Thread 0 sinaliza que encontrou a senha, as outras threads já haviam realizado milhares de tentativas em suas próprias partições antes de receberem o sinal de parada, inflando a soma total de tentativas, mas reduzindo drasticamente o tempo de relógio absoluto.

3. **Casos de Falha (Teste 5):** Quando a senha não existe no espaço de busca gerado, ambas as aplicações (Sequencial e Paralela) executam exatamente o mesmo número máximo de tentativas (114.500.000), garantindo a corretude e integridade da varredura, mas a versão paralela finaliza a varredura quase 4 vezes mais rápido.

## 5. ATIVIDADES POR PARTICIPANTES

- **Gabriel Saraiva:** Responsável pelo desenvolvimento sequencial, lógica de ataque por dicionário, e mutações por sufixo e substituição.
- **Olga Maria:** Responsável pelo desenvolvimento sequencial, estruturação da função de ataque por mutação e mutação por concatenação.
- **Yasser Farid:** Responsável pelo desenvolvimento sequencial, estruturação do ataque numérico exaustivo e lógica de mutação por data.

## 6. AUTO AVALIAÇÃO

**O que funciona:** A arquitetura de fallback (Mutação -> Dicionário -> Números) executando de forma sequencial com paralelismo interno se provou muito estável. O particionamento dos loops entre as threads é preciso. O filtro do tamanho da senha no momento da leitura do arquivo otimiza drasticamente o uso da memória RAM.

**Dificuldades:** A maior complexidade do projeto foi garantir que as threads parassem imediatamente sua execução paralela assim que a senha fosse encontrada. O gerenciamento de ponteiros compartilhados (a variável `senha_achada`) exigiu rigor para evitar acessos irregulares de memória (*segfaults*).

### Limitações Atuais:

- A aplicação carrega o arquivo de dicionário inteiro na RAM de forma prévia (o que esgotaria a memória para arquivos de dezenas de Gigabytes).
- A necessidade de limpar a memória (free) durante a execução limita a performance bruta.
- O tipo de senha restringe-se a letras minúsculas e números.
- A quantidade de mutações aplicadas foi limitada a 1.000.000 (sendo 250 mil para cada tipo) para evitar processamento infinito.
- O algoritmo de mutação foi concebido operando sempre na perspectiva de tamanho máximo de 10 dígitos, enquanto o tempo de processamento viabilizou testes de até 12 caracteres.

**Observação sobre a Corretude:** Considerando que a premissa central do projeto é a descoberta de uma senha específica, o grupo avaliou que a métrica de sucesso (o output da senha correta e a finalização da busca) atua como prova de conceito. Devido a essa natureza do problema de força bruta, optamos por focar na otimização das lógicas de varredura em detrimento de uma função passo-a-passo exclusiva para comparação de corretude.